# DSA Arrays
## Complete Pattern Notes
*Self-Study & Last-Minute Revision Guide*

---

# Pattern Recognition Master Guide

Run through these checks in order when you see a new problem.

### Step 1 — Read Output Type First

| Output Type | Pattern Signal |
|---|---|
| Single number (max, min, count) | DP, Kadane, Sliding Window |
| Index or position | Binary Search, Two Pointer |
| Subarray or substring | Sliding Window, Prefix Sum, Kadane |
| All combinations or subsets | Backtracking |
| Yes or No | BFS, DP, Union-Find |
| All unique triplets or quadruplets | Two Pointer + Sorting |

### Step 2 — Read Constraints

| Constraint | Max Complexity | Pattern Family |
|---|---|---|
| n <= 20 | O(2^n) | Backtracking, Bitmask DP |
| n <= 1,000 | O(n^2) | Double loops fine |
| n <= 100,000 | O(n log n) | Binary Search, Sorting, Heap |
| n <= 1,000,000 | O(n) | Sliding Window, Two Pointer, Prefix Sum |
| n <= 1,000,000,000 | O(log n) | Pure Binary Search, Math |

### Step 3 — Keyword Cheat Sheet

| Keyword in Problem | Pattern to Use |
|---|---|
| subarray, substring, window | Sliding Window |
| sorted + pairs, triplets | Two Pointer |
| range sum, range query | Prefix Sum |

| Keyword in Problem | Pattern to Use |
|---|---|
| maximum subarray sum | Kadane's Algorithm |
| next greater, previous smaller | Monotonic Stack |
| minimum maximum / maximum minimum | Binary Search on Answer Space |
| in-place, rearrange, partition | Two Pointer (fast/slow) |
| sort 0s 1s 2s, 3 categories | Dutch National Flag |
| top K, kth largest | Heap |
| all paths, combinations, subsets | Backtracking |

## Subarray Decision Tree

| Situation | Pattern |
|---|---|
| All positive + subarray sum condition | Sliding Window |
| Negatives + sum EQUALS K | Prefix Sum + Hashmap |
| Negatives + sum LESS / GREATER K | Prefix Sum + Binary Search |
| Max or min subarray sum | Kadane's Algorithm |
| Fixed window size given | Sliding Window (fixed) |
| Count subarrays satisfying condition | Prefix Sum + Hashmap |

## Two Pointer vs Sliding Window vs Hashmap

| | Two Pointer | Sliding Window | Hashmap |
|---|---|---|---|
| Looks for | Pairs or groups | Subarrays | Past occurrences |
| Sorted needed | Usually yes | Usually no | No |
| Key property | Monotonic movement | Window condition | O(1) lookup |
| Negatives | Breaks sum problems | Breaks variable | Works fine |

# Pattern 1 — Two Pointer

*Core Idea: Two variables scanning the structure. Moving one pointer provably eliminates a set of answers.*

## The 3 Flavors

| Flavor | Direction | Used For | Signal Words |
|---|---|---|---|
| Opposite Ends | L→ ←R | Sorted arrays, pairs, triplets | find pair summing to X, max water |
| Same Direction | slow→ fast→ | In-place modification, duplicates | in-place, remove duplicates, move zeros |
| Two Arrays | i→ j→ | Merge, compare two sorted arrays | two sorted arrays, merge, intersection |

## Recognition Check (3 steps)

- Step 1: Do I need elements that TOGETHER satisfy a condition? No → unlikely
- Step 2: Is there sorted order or can I sort without losing info? No → unlikely
- Step 3: Does moving a pointer make condition provably better or worse? No → won't work

## Duplicate Skipping Rule

| Pointer | Guard | Condition |
|---|---|---|
| i (outer loop) | i > 0 | nums[i] == nums[i-1] |
| j (inner loop) | j > i+1 | nums[j] == nums[j-1] |
| left / right | left < right | Skip after recording answer, then move BOTH |

*Key Rule: Compare BACKWARD (i vs i-1), not forward. After recording answer — always move BOTH pointers (left++ and right--).*

## Template

```
L = 0, R = n-1
while L < R:
    if condition_met:  → record answer, L++, R--
    elif need_more:    → L++
    else:              → R--
```

## Solved Problems

| Problem | LC# | Key Insight |
|---|---|---|
| Two Sum II | 167 | Sorted — move based on sum vs target |
| 3Sum | 15 | Fix i, two pointer for rest, skip duplicates |
| 4Sum | 18 | Fix i and j, two pointer for rest |

| Problem | LC# | Key Insight |
|---|---|---|
| Container With Most Water | 11 | Move shorter pointer — monotonic guarantee |
| Trapping Rain Water | 42 | Move pointer where max is smaller, O(1) space |
| Move Zeroes | 283 | slow = next non-zero position |
| K-diff Pairs | 532 | Better with hashmap — k=0 edge case is tricky |

# Pattern 2 — Sliding Window

*Core Idea: Maintain a window and update only what changed. Reuse previous computation instead of recomputing from scratch.*

## Two Flavors

| Flavor | Window Size | Signal Words | Record Answer |
|---|---|---|---|
| Fixed | Given as K | subarray of size K, every window of K | Outside while |
| Variable | Condition-based | longest, smallest, at most K, at least K | Inside while (min) / Outside (max) |

## Fixed Window Template

```
for right in range(n):
    add nums[right] to window
    if right >= k-1:
        record answer
        remove nums[left]; left++
```

## Variable Window Template

```
for right in range(n):
    add nums[right] to window state
    while window VIOLATES condition:   ← always WHILE not IF
        remove nums[left]; left++
    record answer here                 ← outside = maximum, inside = minimum
```

## Critical Rules

| Rule | Why |
|---|---|
| Always WHILE not IF for shrinking | After shrinking once, window might still be invalid |
| Record INSIDE while for min length | Want smallest valid window — shrink as much as possible |
| Record OUTSIDE while for max length | Want largest valid window — after window is valid |
| Erase from map when frequency hits 0 | Otherwise map.size() gives wrong distinct count |
| Negatives break variable window | Shrinking doesn't predictably decrease sum |

## Solved Problems

| Problem | LC# | Type | Key Insight |
|---|---|---|---|
| Max Average Subarray I | 643 | Fixed | Simple add/remove |
| Longest Substring No Repeat | 3 | Variable+Map | Shrink until duplicate gone |

| Problem | LC# | Type | Key Insight |
|---|---|---|---|
| Minimum Size Subarray Sum | 209 | Variable | Record inside while, all positives |
| Permutation in String | 567 | Fixed+Map | Window size = len(s1), match freq maps |
| Minimum Window Substring | 76 | Variable+Map | formed counter for O(1) validity check |

# Pattern 3 — Prefix Sum

*Core Idea: Precompute cumulative sums so any range query is O(1). sum(L,R) = prefix[R+1] - prefix[L]*

## Building Prefix Array

```
vector<int> prefix(n+1, 0);                // 1-indexed avoids L-1 edge case
for(int i = 0; i < n; i++)
    prefix[i+1] = prefix[i] + nums[i];
// sum(L, R) = prefix[R+1] - prefix[L]
```

## Key Rules

| Rule | Why / How |
|---|---|
| Always init freq[0]=1 or index[0]=-1 | Handles subarrays starting at index 0. Without it, miss prefix[R]==K cases. |
| Negative remainder fix: ((x%k)+k)%k | C++ % returns negative for negative numbers. +k then %k makes it positive. |
| Store first occurrence for length | First occurrence of prefix sum gives maximum subarray length. |
| Store frequency for count | Frequency of prefix sum gives count of valid subarrays. |

## Key Formulas

| Problem Type | Formula | Map Stores |
|---|---|---|
| Subarray sum equals K | prefix[L-1] = prefix[R] - K | prefix_sum → frequency |
| Subarray sum multiple of K | prefix[R]%K == prefix[L]%K | remainder → first_index |
| Longest subarray sum = K | prefix[R] - K seen before current R | prefix_sum → first_index |
| Count subarrays div by K | same remainder seen before | remainder → frequency |

## Solved Problems

| Problem | LC# | Key Insight |
|---|---|---|
| Subarray Sum Equals K | 560 | freq[0]=1, lookup prefix-K in map |
| Continuous Subarray Sum | 523 | Store remainders+indices, check length >= 2, first occurrence only |
| Subarray Sums Divisible by K | 974 | freq[0]=1, handle negatives with ((x%k)+k)%k |

# Pattern 4 — Kadane's Algorithm

*Core Idea: At every index, decide — extend current subarray OR start fresh. Negative running sum only hurts what comes next, so discard it.*

## The Decision at Every Index

```
current = max(nums[i], current + nums[i])   // extend or restart
answer  = max(answer, current)
```

```
// Always start with nums[0], not 0
// If all negative, answer is least negative — not 0
```

## When to Use Kadane's

| Use Kadane's | Do NOT Use Kadane's — Use Instead |
|---|---|
| Maximum subarray sum | Count subarrays with sum = K → Prefix Sum + Hashmap |
| Minimum subarray sum | Exact sum equals K → Prefix Sum + Hashmap |
| Maximum subarray product | Longest subarray with sum <= K → Sliding Window |
| Circular array max/min | Non-contiguous elements → DP |

## Subarray vs Subsequence vs Subset

| Keyword | Meaning | Pattern |
|---|---|---|
| Subarray | Contiguous, no gaps — elements next to each other | Kadane's or Sliding Window |
| Subsequence | Non-contiguous, can skip elements, order maintained | DP |
| Subset | Non-contiguous, any order, all combinations | DP or Backtracking |

## Variants

### Variant 1 — Maximum Product Subarray (LC 152)

**Key difference:** Track BOTH max and min at every index. Negative × negative = positive, so min can flip to become max.

```
int tempMax = maxProd;   // save before overwriting
maxProd = max({nums[i], maxProd*nums[i], minProd*nums[i]});
minProd = min({tempMax*nums[i], minProd*nums[i], nums[i]});
```

### Variant 2 — Circular Subarray (LC 918)

**Two cases:** (1) No wrap → regular Kadane's. (2) Wraps → totalSum - minSubarraySum.

```
return maxSum > 0 ? max(maxSum, totalSum - minSum) : maxSum;
// Edge case: all negative → totalSum-minSum=0 (wrong). Return maxSum.
```

## Solved Problems

| Problem | LC# | Key Insight |
|---|---|---|
| Maximum Subarray | 53 | Basic Kadane's, start with nums[0] |
| Maximum Sum Circular Subarray | 918 | Run max AND min Kadane's together, handle all-negative edge case |
| Maximum Product Subarray | 152 | Track both max and min, use tempMax to avoid overwrite bug |

# Pattern 5 — Monotonic Stack

*Core Idea: Stack maintaining increasing or decreasing order. When new element violates order, pop — each popped element found its answer at exactly that moment.*

## Two Types

| Type | Order (bottom→top) | Used For | Pop When |
|---|---|---|---|
| Increasing Stack | Increases | Next/Prev SMALLER element | New element is smaller |
| Decreasing Stack | Decreases | Next/Prev GREATER element | New element is larger |

*Time Complexity: O(n) — each element is pushed and popped exactly once.*

## Template — Always Store Indices

```
stack<int> st;             // store INDICES not values
for(int i = 0; i < n; i++){
    while(!st.empty() && nums[st.top()] < nums[i]){
        int idx = st.top(); st.pop();
        answer[idx] = nums[i];   // nums[i] is next greater for idx
    }
    st.push(i);
}
// remaining in stack have no next greater → answer stays -1
```

*Why store indices: You need position for width calculations and placing answers. Values can always be retrieved from indices.*

## Largest Rectangle — Width Calculation

```
int width = st.empty() ? i : i - st.top() - 1;
// Stack empty → extends all way to left → width = i
// Stack not empty → from just after new top to just before i
// Sentinel trick: add height=0 at end to force remaining bars to process
int h = (i == n) ? 0 : heights[i];
```

## Signal Words

- next greater / next smaller element
- previous greater / previous smaller element
- largest rectangle in histogram
- buildings with ocean view, stock span, sum of subarray minimums

## Solved Problems

| Problem | LC# | Key Insight |
|---|---|---|
| Next Greater Element I | 496 | Build map for nums2, lookup nums1 elements |
| Daily Temperatures | 739 | Store distance i-idx instead of value. Init res with 0s — no cleanup needed |
| Largest Rectangle in Histogram | 84 | Pop when shorter found, width = i - st.top() - 1, sentinel 0 at end |

# Pattern 6 — Dutch National Flag

*Core Idea: Partition array into 3 sections in single O(n) pass using 3 pointers. Only works when exactly 3 distinct value types exist.*

## The 3 Pointers

- low — everything before low is section 1 (smallest)
- mid — current element being processed
- high — everything after high is section 3 (largest)
- Region between low and high is unknown/unsorted

## Template

```
int low=0, mid=0, high=n-1;
while(mid <= high){
    if(arr[mid] == 0){
        swap(arr[low], arr[mid]);
        low++; mid++;          // both move — low element is processed
    } else if(arr[mid] == 1){
        mid++;                 // already in right place
    } else {
        swap(arr[mid], arr[high]);
        high--;                // do NOT mid++ — swapped element is unknown
    }
}
```

## Critical Rules

| Rule | Why |
|---|---|
| Do NOT increment mid when swapping with high | Element coming from high is unknown — hasn't been examined yet |
| DO increment mid when swapping with low | Element from low is already processed (was 0 or 1) |
| Loop condition is mid <= high | mid can equal high — that element still needs processing |

## Connection to QuickSort

**3-way partition in QuickSort = Dutch National Flag around a pivot.** Makes QuickSort O(n) on arrays with many duplicates (instead of O(n^2)).

## Solved Problems

| Problem | LC# | Variant |
|---|---|---|
| Sort Colors | 75 | Direct DNF with 0, 1, 2 |

| Problem | LC# | Variant |
|---|---|---|
| Sort Array By Parity | 905 | 2-section — even left, odd right |
| Move Zeroes | 283 | Fast/slow pointer — maintain relative order of non-zeros |

# Pattern 7 — Binary Search Variants

*Core Idea: Eliminate half the search space at each step. Works on any monotonic structure — not just sorted arrays.*

## Two Templates

### Template 1 — Find Exact Value

```
int lo=0, hi=n-1;
while(lo <= hi){
    int mid = lo + (hi-lo)/2;   // lo+(hi-lo)/2 avoids overflow
    if(arr[mid]==target) return mid;
    else if(arr[mid]<target) lo=mid+1;
    else hi=mid-1;
}
return -1;
```

### Template 2 — Find Boundary (First True / Last False)

```
int lo=0, hi=n-1, ans=-1;
while(lo <= hi){
    int mid = lo + (hi-lo)/2;
    if(condition(mid)){
        ans=mid;
        hi=mid-1;   // first true (leftmost) — use lo=mid+1 for last true
    }
    else lo=mid+1;
}
return ans;
```

## lo <= hi vs lo < hi

| Use | When |
|---|---|
| lo <= hi (with ans variable) | Finding exact value or boundary with explicit tracking |
| lo < hi (return lo at end) | Minimization or maximization — lo == hi IS the answer |

## Key Variants

| Variant | Key Insight |
|---|---|
| First & Last Position (LC 34) | First: record ans, go LEFT. Last: record ans, go RIGHT. Same template, different direction. |
| Rotated Sorted Array (LC 33) | One half is ALWAYS sorted. Use sorted half to decide which side target is in. |

| Variant | Key Insight |
|---|---|
| Binary Search on Answer (LC 875) | Search over possible ANSWERS not array indices. Verify candidate in O(n). lo=min ans, hi=max ans. |
| Min in Rotated Array (LC 153) | Compare mid with hi (not lo). If nums[mid]>nums[hi] → min is in right half. |

## Answer Space Binary Search — Pattern

- Signal words: minimum maximum, maximum minimum, smallest K such that...
- Step 1: Define search space [lo, hi] — range of possible answers
- Step 2: Define condition(mid) — can I verify if mid is valid in O(n)?
- Step 3: Binary search for boundary. If valid → try smaller (hi=mid). If not → lo=mid+1.

## Solved Problems

| Problem | LC# | Key Insight |
|---|---|---|
| Find First and Last Position | 34 | Separate functions for first and last, same template |
| Search in Rotated Array | 33 | Identify sorted half, check if target falls within it |
| Koko Eating Bananas | 875 | lo=1, hi=max(piles), canFinish() verifies in O(n), ceil division |
| Find Minimum in Rotated | 153 | Compare with hi not lo, use lo < hi template |

# Hashmap — When to Use

*Core Idea: Remember something you saw before and look it up in O(1).*

## The 4 Signals

| Signal | Example |
|---|---|
| Check if something EXISTS in past | does complement exist, have I seen this character |
| COUNT frequencies | how many times does X appear, first non-repeating char |
| STORE and RETRIEVE linked values | two sum — store index with number |
| Find PAIR without sorting | find two numbers summing to target in unsorted array |

## Two Pointer vs Hashmap Boundary

| Situation | Use |
|---|---|
| Sorted array + pairs | Two Pointer |
| Unsorted array + pairs | Hashmap |
| Need index information | Hashmap |
| Monotonic property exists | Two Pointer |
| Frequency counting needed | Hashmap |

*One Line Rule: If sorting would lose important info (like indices) OR no monotonic property exists → hashmap over two pointer.*

# Quick Reference — All 7 Patterns

| Pattern | Time | Space | Use When |
|---|---|---|---|
| Two Pointer | O(n) | O(1) | Sorted + pairs/groups + monotonic property |
| Sliding Window | O(n) | O(k) | Contiguous subarray + window condition |
| Prefix Sum | O(n) | O(n) | Range queries, count/exact subarray sums |
| Kadane's | O(n) | O(1) | Max or min of contiguous subarray |
| Monotonic Stack | O(n) | O(n) | Next/prev greater/smaller, histogram area |
| Dutch Nat. Flag | O(n) | O(1) | 3 distinct values, in-place partition |
| Binary Search | O(log n) | O(1) | Sorted array or monotonic answer space |

## Most Important Rules to Remember

- Two Pointer: compare BACKWARD for duplicates. Move BOTH pointers after match.
- Sliding Window: always WHILE not IF. Record inside while = minimum, outside = maximum.
- Prefix Sum: always init freq[0]=1. Negative remainder fix: ((x%k)+k)%k.
- Kadane's: start with nums[0] not 0. Track both max AND min for product variant.
- Monotonic Stack: store INDICES not values. Each element pushed and popped exactly once.
- Dutch National Flag: do NOT increment mid when swapping with high.
- Binary Search: mid = lo+(hi-lo)/2 to avoid overflow. lo<hi when answer = lo at end.

*— End of Notes —*